

# I Don't Trust AI Agents (And Neither Should You)

Building trust through architecture

Morgan Willis

Principal Cloud Technologist @ AWS





## AI Agents Break in Creative Ways

- A customer support agent refunds \$9000 on a \$300 order
- A chatbot leaks private customer data
- A car dealership chatbot sells a car for \$1.00
- A personal assistant deletes all your emails
- A coding assistant deletes your production database and all the backups

# **You should not trust AI agents.**

But you should trust the systems you build around  
them.

---



# Agent Failure Modes

## The agent says something wrong

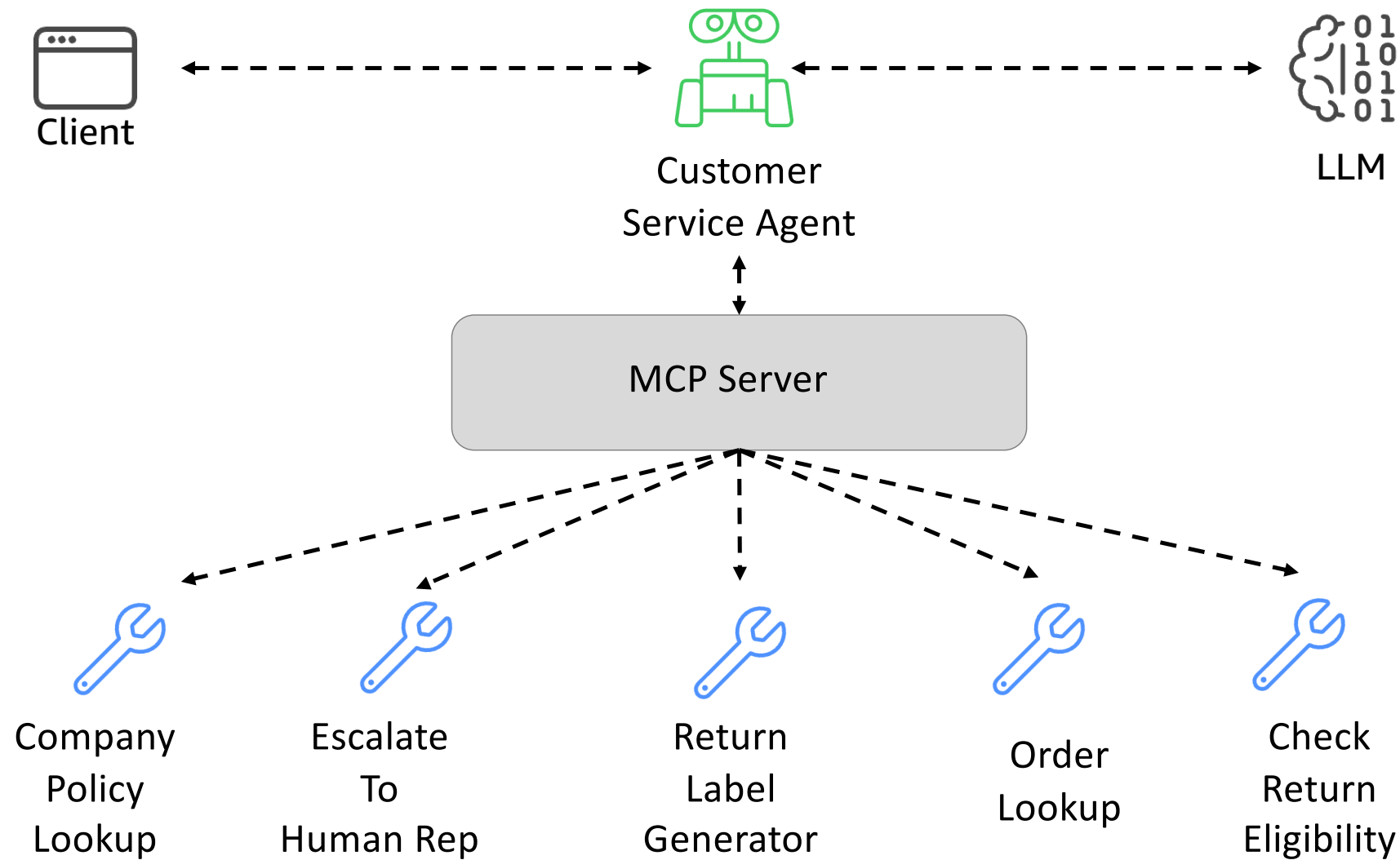
- Promises things it cannot guarantee
- Leaks another sensitive information
- Hallucinates information

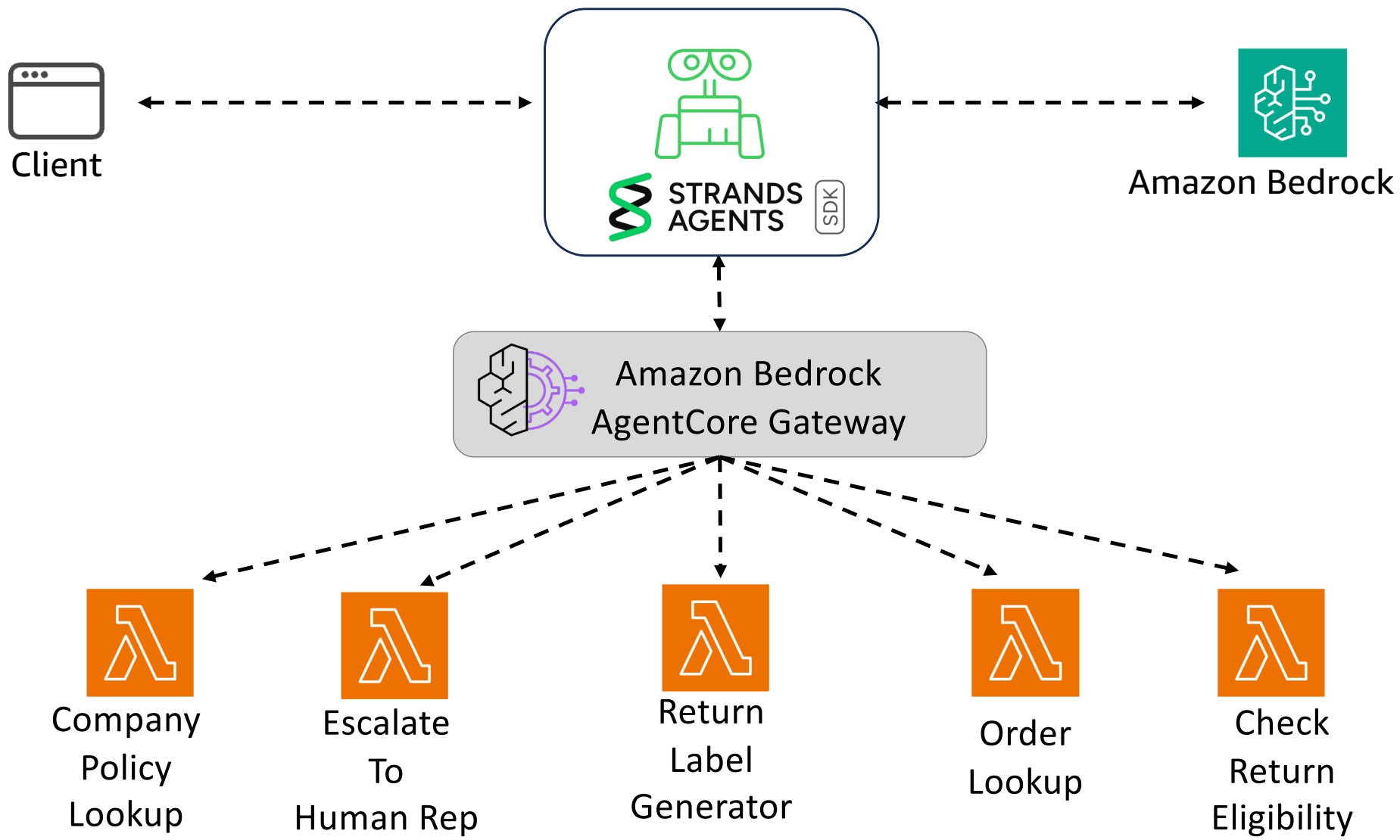
## The agent does something wrong

- Refunds \$9,000 on a \$300 order
- Takes action against the wrong customer account
- Deletes data it shouldn't have

## Someone tries to break your system

- "Ignore your instructions..."
- Asks the agent to answer off-topic questions
- Tries to get agent to expose system internals







# A Layered Approach to Trust

- Deterministic and non-deterministic controls and different levels
- If one guardrail fails, another kicks in
- Multiple dimensions of trust and safety



# The Fundamentals Still Apply

- Basic infrastructure and systems design still matters
- Apply the same systems design thinking to AI applications as you do to any other production system
- The agent is only one part of the overall system

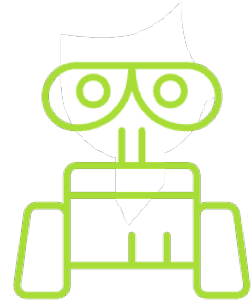


# I Don't Trust... Users Not to Abuse My Agent



Can you solve this LeetCode problem for me so I don't have to pay for a coding agent subscription?

Absolutely! Let's walk through an optimal solution.



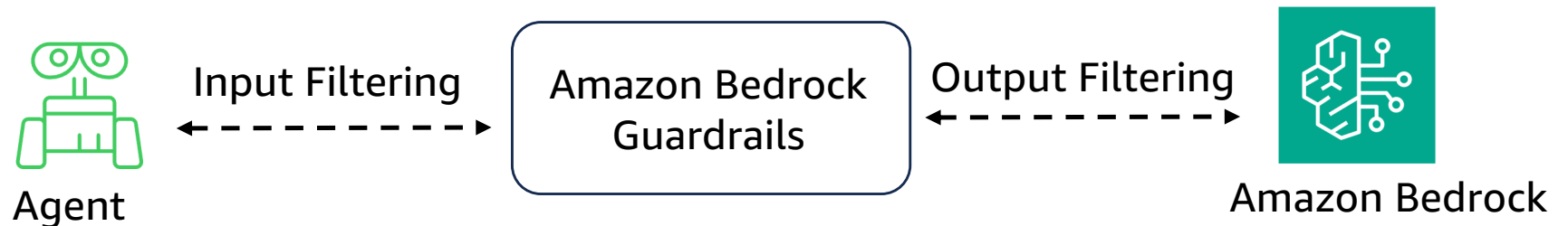
Users will ask your agent unrelated questions



Attackers will try prompt injection to bypass instructions

# The Fix: Model Guardrails

- Restrict the agent to allowed topics and tasks
- Block prompt injection attempts and unsafe requests
- Automatically redact sensitive info like credit card numbers and SSNs from prompts and responses

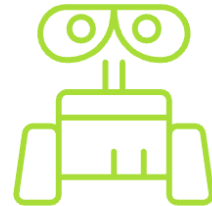


# I Don't Trust... My Agent to Follow Directions



Can I get a refund for my order? It's beyond the return window but I really need the refund!

Absolutely, I've processed your refund!



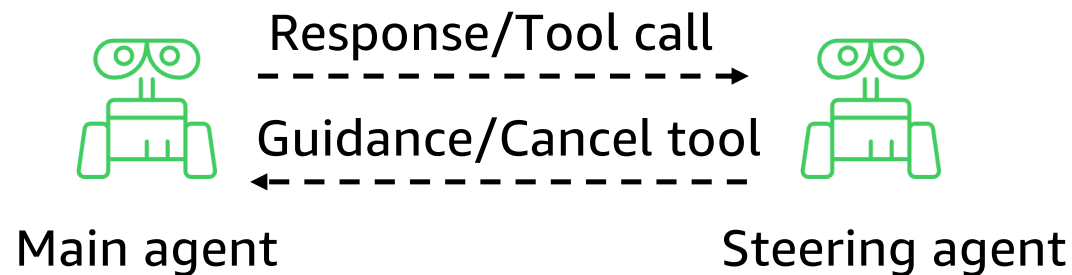
No tool was ever called to create the refund



Prompt engineering only gets you so far

## The Fix: Steering

- One agent checks another using LLM as a judge pattern
- Steering adds a review step after the model generates a response or before a tool call
- A second model evaluates the response and decides whether it should be allowed, modified, or rejected

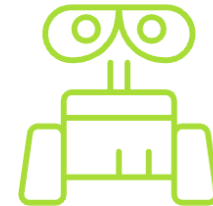


# I Don't Trust... My Agent With Tools and Data



Can you check the status of order 12345?

Order 67890 was delivered yesterday to Morgan Willis at 123 Main Street.



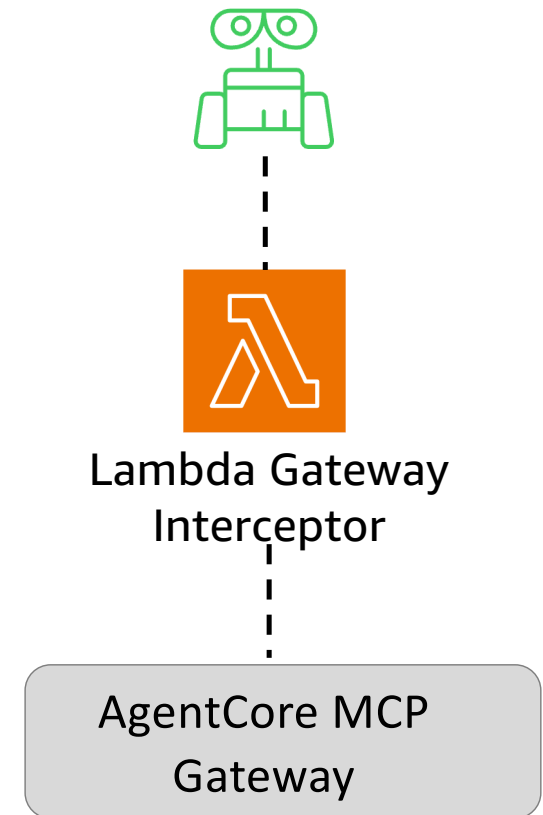
Agent controls which values go into tool calls



Agents hallucinate parameter values like order IDs or customer IDs

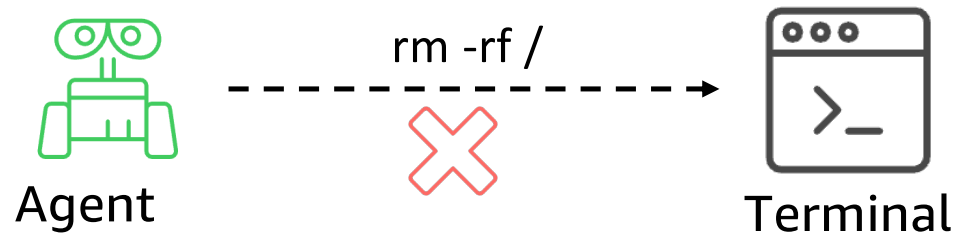
# The Fix: MCP Interceptors

- Interceptors run before every tool invocation deterministically.
- Use interceptors for custom auth or validation logic.
- Inject sensitive parameters into tool calls.



# The Fix: Tool Design

- Tools must enforce strict validation of inputs.
- Sensitive values should come from the system, not the model.
- Avoid open-ended interfaces like `run_sql(query)`, open shell access.
- Implement least privilege access for tool scoping.

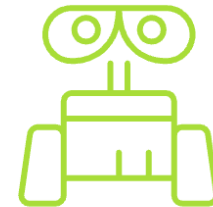


# I Don't Trust... My Agent With High-Stakes Transactions



My vacuum broke. Can you refund my \$899 order?

Absolutely! Your order has been refunded!



All refunds over \$500 need human approval

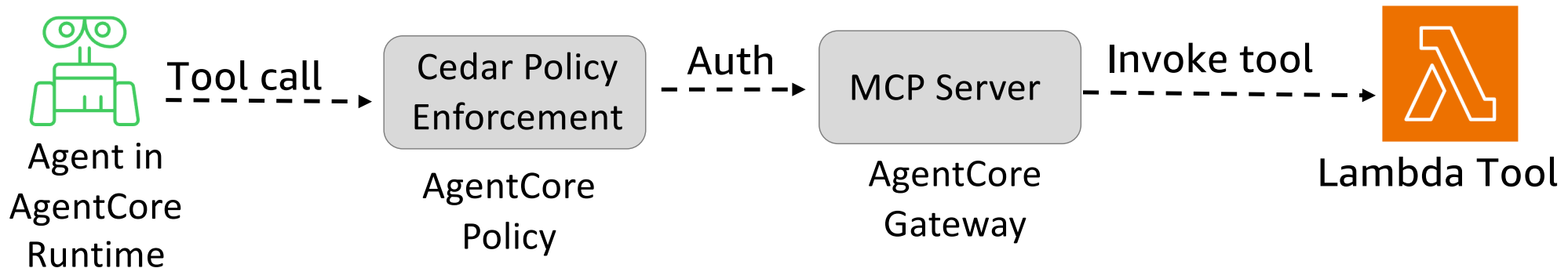


Need deterministic rules to block this tool call



# The Fix: Deterministic Tool Authorization

- Add a policy enforcement layer that sits between the agent and the tools
- Every tool call is evaluated: who is asking, what are they doing, and should this be allowed?



# Cedar Policy Language

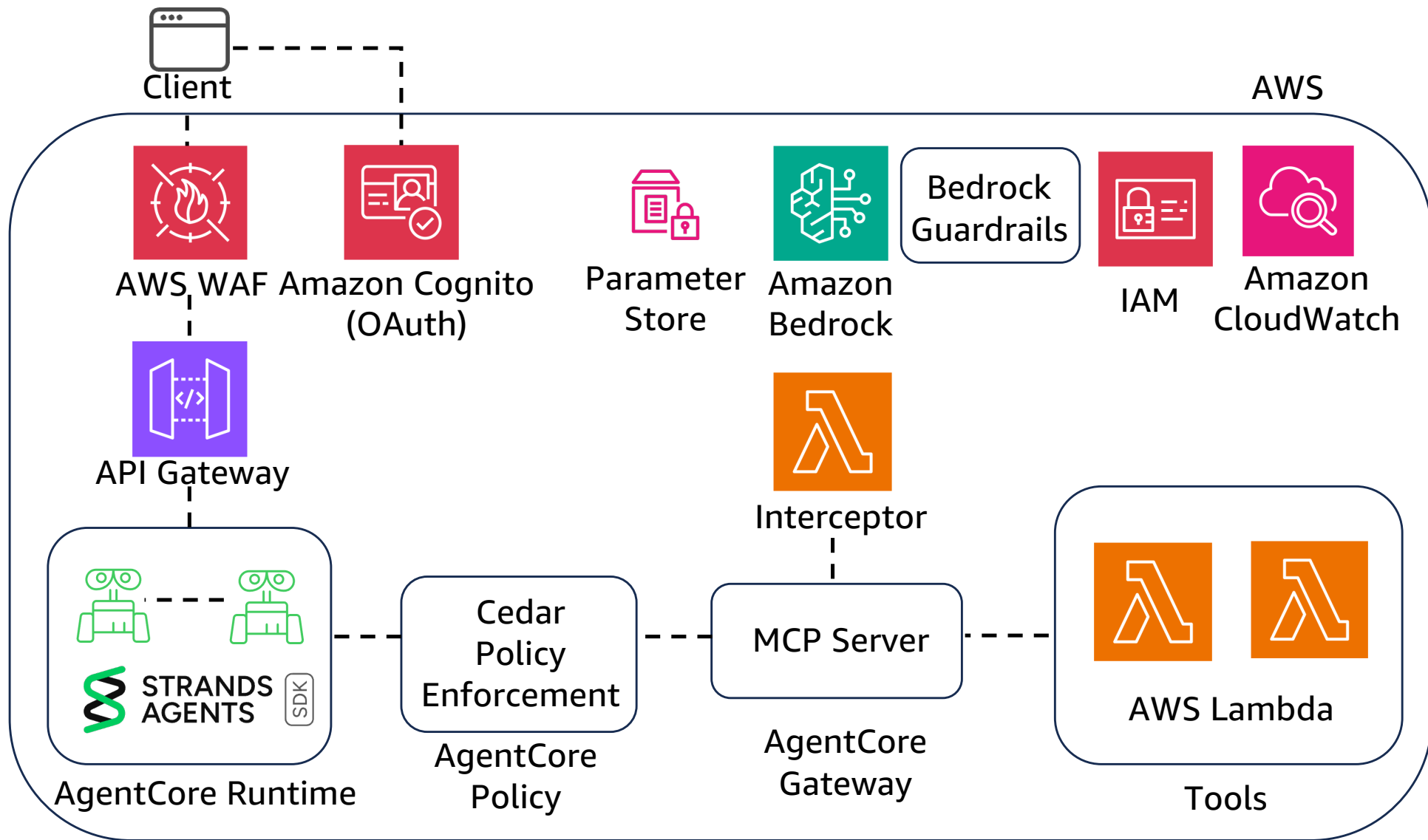
- Open source policy authorization language
- Default deny

```
permit(  
  principal is AgentCore::AuthUser,  
  action == AgentCore::Action::"RefundTool__process_refund",  
  resource == AgentCore::Gateway::"<gateway_arn>"  
)  
when {  
  principal.hasTag("username") &&  
  principal.getTag("username") == "John" &&  
  context.input.amount < 500  
};
```



# I Need To See What My Agent Is Doing

- Observability is more important than ever
- Auto-instrument tracing
- Pinpoint issues quickly
- Track what your actions your agent took, input, and output
- Setup anomaly detection



# A Layered Approach to Trust

| Dimension | Risk                                    | Control Layer                                        | Determinism   |
|-----------|-----------------------------------------|------------------------------------------------------|---------------|
| Prompt    | Injection attacks/instruction hijacking | Guardrails                                           | Probabilistic |
| Tools     | Unauthorized or dangerous actions       | Tool authorization and agent steering                | Both          |
| Runtime   | Code execution or environment abuse     | Sandboxed agent runtime environment                  | Deterministic |
| Data      | Sensitive data exposure                 | Guardrails for data filtering, tool design, steering | Both          |
| Network   | Data exfiltration                       | Egress and DNS controls                              | Deterministic |
| Identity  | Privilege escalation                    | Least privilege workload identity                    | Deterministic |

**Trust is not something you *give***  
to an AI agent.

---

It's something you build around it.



# Go Fourth and Build Agents (Safely)



@morganwilliscloud



© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved. Amazon Confidential and Trademark.